

Evaluation Comments Template

General Information

- Evaluator Name: P4
 - Application Name: spring-petclinic
 - Refactoring Approach: microrefact
-

1) Services Boundaries and Decompositions

1. Are all of the classes in the application mapped to their correct microservices based on their decomposition?

R1: No, there is some class that is not mapped to the correct microservice . Many other classes are duplicated across different microservices. The decomposition of microservice 1 proposes 11 classes, but we found 18. For the second microservice, we found 19 classes instead of 10, and for the third microservice, 13 classes instead of 6.

2. Are there classes that were not included in the decomposition but found in the monolith? How were they handled by the approach?

Describe any missing or unaccounted classes:

R2: Yes, the VisitController class does not exist in any microservice, which implies there is no control over adding visits in the application. This disrupts the MVC architecture of the system.

3. Are there new classes added to the microservices? Do they still respect the single responsibility of these microservices?

R3: There are no new classes except for the main class, and this microservice generally adheres to the Single Responsibility Principle.

4. Has the dependency mapping between components been accurately preserved?

R4: Yes, generally the dependency mapping between components has been preserved. I note here that there is not much difference between the monolithic code and the refactored one.

5. How well did the approach handle this aspect? (Score from 1 to 10)

R5: Compared to human refactoring, I would give this refactored application a score of 3 out of 10.

6. Additional Comments

R6: Many classes are duplicated across different microservices, and I don't see any real separation of concerns between them.

2) Correctness & Functional Equivalence:

1. Does the refactored application produce the same outcomes as the original monolith?

R1: Yes, through a manual verification, it seems that the refactored application produces the same outcomes as the original monolith.

2. Have any functional behaviors been inadvertently altered during the transformation?

R2: Yes, the transformation appears to have inadvertently altered the expected behavior by removing or modifying the method that directly handles the display of the visit creation form. While the core functionality of loading the pet and owner data remains intact, the mechanism for rendering the form itself seems to have been impacted. As a result, this change could lead to issues in the user experience, such as missing or incomplete forms being displayed to users, which deviates from the original functionality where users could expect a fully loaded form for creating a visit.

3. Is the transformation strictly limited to modifying call mechanics, leaving the business logic unchanged?

R3: Yes, only minor modifications have been made to the business logic, and the transformation is strictly focused on changing the call mechanics.

4. Do the transformed interactions maintain the same logical flow as in the monolith?

R4: Yes, the business logic was preserved, and the same logical flow was maintained. Most of the classes were duplicated across the different microservices, and the logical flows were also preserved.

5. How well did the approach handle this aspect? (Score from 1 to 10)

R5: If we only talk about Correctness & Functional Equivalence, I would give this approach a score of 5 out of 10.

6. Additional Comments

3) Code Transformation and Remote Invocations:

1. Are all relevant local method calls successfully replaced by their remote call counterparts?

R1: Based on a manual evaluation, the majority of the calls remain preserved within the same microservice due to the duplication of classes between microservices. However, calls between classes in different microservices have been replaced by their remote call counterparts.

2. Are the request and response structures consistent with the original local calls?

R2: Yes, the request and response structures are consistent with the original local calls.

3. Are there any issues such as missing object mappings or incorrect handling of requests and responses?

R3: Yes, several manipulations of non-existent objects exist in the code, especially for objects of classes that no longer exist in the refactored version.

4. Is data correctly serialized and deserialized during remote calls?

R4: Yes, data is correctly serialized and deserialized during remote calls (Ex : BaseEntity.java).

5. How well did the approach handle this aspect? (Score from 1 to 10)

R5: If we only talk about Code Transformation and Remote Invocations, I would give this approach a score of 7 out of 10.

6. Additional Comments

4) Data Handling and Consistency:

1. Is there an existing database in the monolith? How did the refactoring approach handle it?

R1: Yes, in the monolithic Spring PetClinic application, there is a database . However, the refactoring approach did not handle the database part, and the refactored application does not contain any database.

2. How is a shared state managed or synchronized across the services?

R2: In a microservices architecture, shared state across services is typically synchronized through API calls. Each service maintains its own data, and when one service needs to interact with another, it makes API requests to update or retrieve shared data

3. Is there any data duplication? Do you think that it is justified?

R3: In this example, I noticed a lot of duplicated code, but there is no duplicated data.

4. Is the data still consistent? Were there any new consistency mechanisms added if needed?

R4: No answer.

5. How well did the approach handle this aspect? (Score from 1 to 10)

R5: I give this approach a 4 out of 10.

6. Additional Comments

R6: The method used by this approach to manage synchronization and shared state across the services is effective. However, I didn't see any solution for handling the associated databases. Because of this, I give a score of 4 out of 10 for this approach, as mentioned in the previous question.

5) Performance Overhead:

1. Are there new mechanisms introduced that add overhead to existing interactions?

R1: An API call was created for each method call to classes that are in other services, which may introduce an overhead on the application.

2. Do you think that this overhead is justified?

R2: Yes, the overhead is definitely justified, and API call communication is the best way to ensure independent and loosely coupled microservices.

3. What mechanisms would you replace if you had to modify the code manually?

R3: Even if I modify the code manually, I will use an API call to represent interactions with classes in other services.

4. Are there any indications of over-segmentation leading to excessive inter-service communication?

R4: No, there are no clear indications of over-segmentation leading to excessive inter-service communication. However, I have noticed code duplication, which could negatively impact long-term maintainability.

5. How well did the approach handle this aspect? (Score from 1 to 10)

R5: I rate this approach 8 out of 10 for this aspect.

6. Additional Comments

6) Code Maintainability & Readability Post-Transformation:

1. Is the refactored code easy to understand and follow compared to the original monolith?

R1: There isn't a significant difference between the refactored microservices code and the original monolith in terms of clarity. Both implementations share similar structures, with the microservices still containing a lot of redundancy across services. Therefore, while the microservices architecture might provide scalability benefits, the ease of understanding and following the code remains largely the same as the original monolithic structure.

2. Has the transformation introduced excessive boilerplate code or complexity?

R2: I didn't see any difference in the coding aspect between the monolith and the refactored version.

3. How difficult is it to replace the mechanisms introduced by the approach if needed?

R3: The maintainability and post-transformation processes are relatively difficult due to class duplication between microservices, and any modification should be made in more than one microservice.

4. Does the approach include fault tolerance or error handling mechanisms?

R4: I didn't find that the approach includes any fault tolerance or error handling mechanisms.

5. How well did the approach handle this aspect? (Score from 1 to 10)

R5: I rate this approach 3 out of 10 for this aspect. The redundancy of classes between microservices makes readability and maintainability more difficult, especially if a local class is updated in one microservice but not in another. After a long time, we may find the same class under the same name in different microservices with different code.

6. Additional Comments

7) Execution and Packaging:

1. Does the refactored version compile and/or package correctly?

R1: Many errors occurred when I tried to compile the project. A lot of nonexistent methods and classes were called.

2. Does the refactoring approach introduce new dependencies?

R1: No answer

3. Does the approach introduce any new deployment automation mechanisms?

R1: This approach does not introduce any new deployment automation mechanisms.

4. How well did the approach handle this aspect? (Score from 1 to 10)

R1: After reviewing the code, it appears well-written and should compile correctly. For that reason, I give it a 3 out of 10 for this question.

5. Additional Comments

8) Additional comments that do not fall into any aspect:

I want to mention once again that the majority of the classes are duplicated across all the microservices, which gives the impression that there is repetition of code rather than refactoring in this approach.